

```

import cv2                                                    # Import OpenCV for image processing algorithms
import numpy as np                                            # Import Numpy for mathematical matrix operations
import matplotlib.pyplot as plt                               # Import Matplotlib to visualize the output on screen

# =====
# STEP 1: LOW-LEVEL CV - Setup Synthetic Input & Apply Noise Filtering
# =====

# Create a synthetic empty background scene (100x100 matrix)
bg = np.zeros((100, 100), dtype=np.uint8)

# Create an incoming frame containing an intruder object (a white square)
raw_frame = bg.copy()
cv2.rectangle(raw_frame, (40, 40), (60, 60), 255, -1)

# Apply Gaussian Blur (Low-Level CV primitive to eliminate high-frequency sensor noise)
filtered_bg = cv2.GaussianBlur(bg, (5, 5), 0)
filtered_frame = cv2.GaussianBlur(raw_frame, (5, 5), 0)

# =====
# STEP 2: MID-LEVEL CV - Background Subtraction & Shape Segmentation
# =====

# Initialize the Gaussian Mixture-based Background/Foreground Segmentation algorithm
backSub = cv2.createBackgroundSubtractorMOG2()

# Feed the historical baseline scene to train the structural background model
backSub.apply(filtered_bg)

# Isolate pixels that deviate from the background model to generate a raw motion mask
motion_mask = backSub.apply(filtered_frame)

# Clean structural fragmentation using mathematical morphology (dilation)
clean_mask = cv2.dilate(motion_mask, None, iterations=1)

# =====
# STEP 3: HIGH-LEVEL CV - Structural Feature Analysis & Threat Trigger
# =====

# Extract distinct topological boundaries (contours) from the isolated binary mask
contours, _ = cv2.findContours(clean_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Create a canvas copy for rendering semantic high-level bounding metrics
output_feed = cv2.cvtColor(raw_frame, cv2.COLOR_GRAY2RGB)

for contour in contours:
    # Filter out insignificant moving fragments (e.g., noise artifacts)
    if cv2.contourArea(contour) > 50:

        # Calculate spatial coordinates of the object bounding box wrapper
        x, y, w, h = cv2.boundingRect(contour)

        # High-Level Decision: Structural classification and alert annotation
        cv2.rectangle(output_feed, (x, y), (x + w, y + h), (255, 0, 0), 1)
        cv2.putText(output_feed, "INTRUSION DETECTED", (5, 15),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.3, (255, 0, 0), 1)

# =====
# STEP 4: VISUALIZATION - Plotting Pipeline Stages side-by-side
# =====

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Plot 1: The Raw Incoming Frame
axes[0].imshow(raw_frame, cmap='gray')
axes[0].set_title('Input: Raw Surveillance Frame')
axes[0].axis('off')

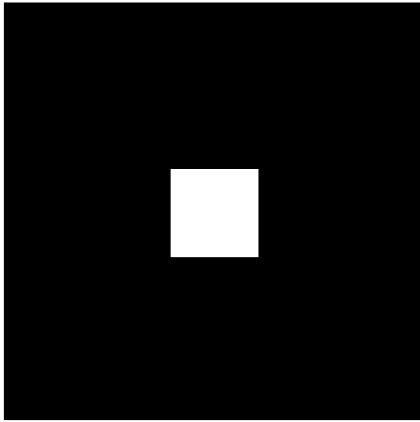
# Plot 2: Mid-Level Extraction
axes[1].imshow(clean_mask, cmap='gray')
axes[1].set_title('Mid-Level: Extracted Motion Mask')
axes[1].axis('off')

```

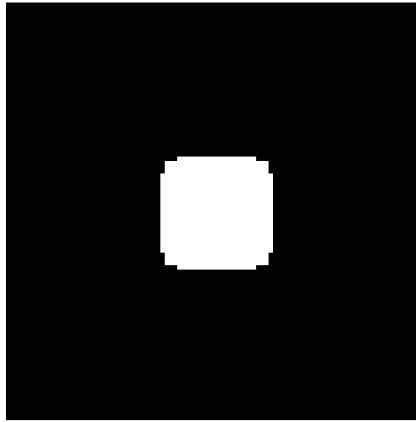
```
# Plot 3: High-Level Automated Threat Verification
axes[2].imshow(output_feed)
axes[2].set_title('High-Level: Semantic Alert Context')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```

Input: Raw Surveillance Frame



Mid-Level: Extracted Motion Mask



High-Level: Semantic Alert Context

